



CSE 711 - Compilers

Lecture 4

Syntax Directed Translation

Ms. Samia Muntaha

Lecturer,

Department of Computer Science and Engineering
University of Chittagong

Outline

- ▶ Semantic Errors
- ▶ Syntax Directed Definition
- ▶ Types of Syntax Trees
- ▶ Construction of AST
- ▶ Type Checking
- ▶ Evaluation Order of SDT

Error Handling

- ▶ Purpose of the compiler is to detect non-valid programs
- ▶ To translate the valid ones
- ▶ Many kinds of possible errors

Error kind	Example (C)	Detected by ...
Lexical	... \$...	Lexer
Syntax	... x *% ...	Parser
Semantic	... int x; y = x(3); ...	Type checker
Correctness	your favorite program	Tester/User

Syntax Error Handling

- Error handler should
 - Report errors accurately and clearly
 - Recover from an error quickly
 - Not slow down compilation of valid code
- Good error handling is not easy to achieve

Syntax Error Recovery

- Approaches from simple to complex
 - Panic mode
 - Error productions
 - Automatic local or global correction
- Not all are supported by all parser generators

Error Recovery: Panic Mode

- Simplest, most popular method
- When an error is detected:
 - Discard tokens until one with a clear role is found
 - Continue from there
- Such tokens are called synchronizing tokens
 - Typically the statement or expression terminators

Error Recovery: Panic Mode (Cont.)

- Consider the erroneous expression
 $(1 + + 2) + 3$
- Panic-mode recovery:
 - Skip ahead to next integer and then continue
- Bison: use the special terminal **error** to describe how much input to skip
 $E \rightarrow \text{int} \mid E + E \mid (E) \mid \text{error int} \mid (\text{error})$

Error Recovery: Error Productions

- Idea: specify in the grammar known common mistakes
- Essentially promotes common errors to alternative syntax
- Example:
 - Write **5 x** instead of **5 * x**
 - Add the production $E \rightarrow \dots \mid E E$
- Disadvantage
 - Complicates the grammar

Error Recovery: Local and Global Correction

- Idea: find a correct “nearby” program
 - Try token insertions and deletions
 - Exhaustive search
- Disadvantages:
 - Hard to implement
 - Slows down parsing of correct programs
 - “Nearby” is not necessarily “the intended” program
 - Not supported by most tools

Syntax Error Recovery: Past and Present

- Past
 - Slow recompilation cycle (even once a day)
 - Find as many errors in one cycle as possible
 - Researchers could not let go of the topic
- Present
 - Quick recompilation cycle
 - Users tend to correct one error/cycle
 - Complex error recovery is less compelling
 - Panic-mode seems enough

Syntax-directed definition (SDD)

- ▶ A syntax-directed definition (SDD) is a context-free grammar with attributes added to the grammar symbols.
 - attributes are stored in the nodes of the syntax tree.
 - each production has a set of semantic rules associated for computing the attributes.
- ▶ If X is a grammar symbol, a is an attribute, then $X.a$ denotes the value of a at node X

Syntax-directed definition (SDD)

- ▶ Example (Syntax-Directed Definitions)

- ▶ Let the grammar be

$$E \rightarrow E + E \mid \text{num}$$

- ▶ Then E derives its value from the num tokens in the expression.

- ▶ This is expressed formally by the rules

$$E.\text{val} = E1.\text{val} + E2.\text{val}$$

$$E.\text{val} = \text{num.lexval}$$

Syntax Directed Translation Scheme

► Grammar

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow \text{id}$

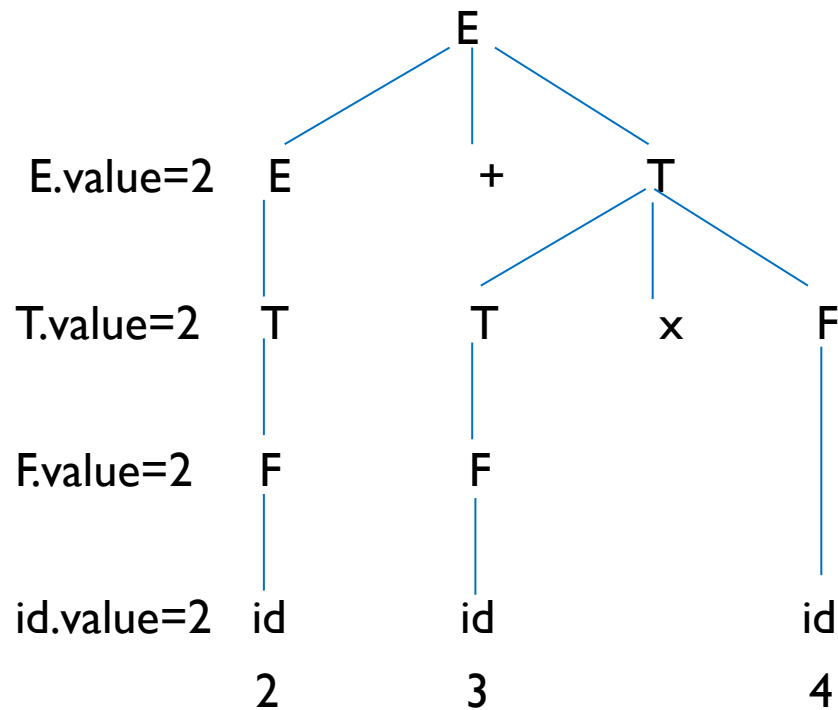
► Semantic Rules

- { $E.\text{value} = E.\text{Value} + T.\text{value}$ }
- { $E.\text{value} = T.\text{value}$ }
- { $T.\text{value} = T.\text{value} \times F.\text{value}$ }
- { $T.\text{value} = F.\text{value}$ }
- { $F.\text{value} = \text{id}.\text{value}$ }

Grammar + Semantic Rules = SDT

Syntax Directed Translation Scheme

Expression: $2 + 3 \times 4 = 14$



▶ Grammar

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow \text{id}$

▶ Semantic Rules

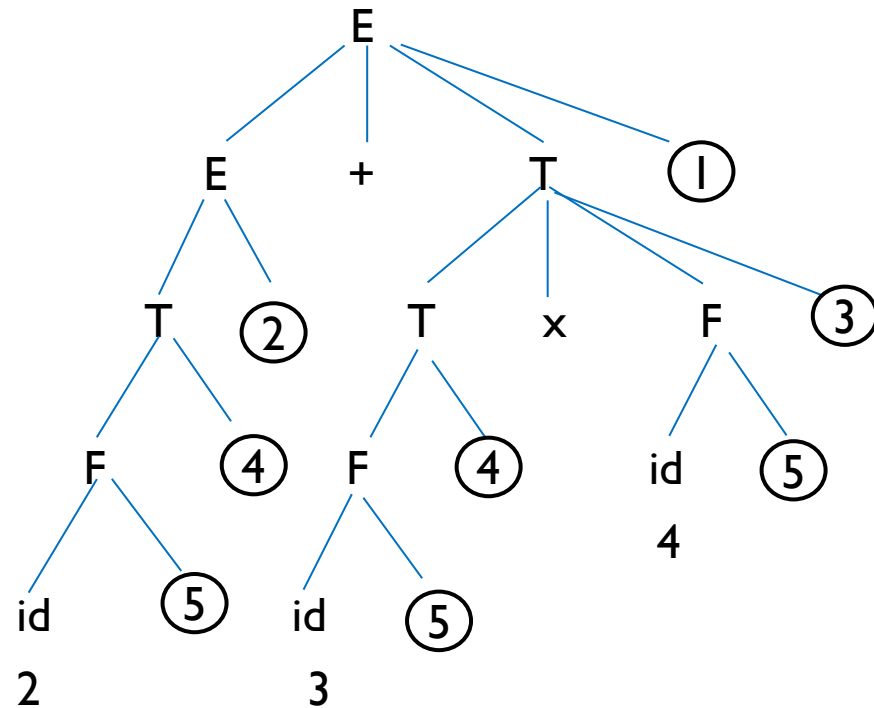
- { $E.\text{value} = E.\text{Value} + T.\text{value}$ }
- { $E.\text{value} = T.\text{value}$ }
- { $T.\text{value} = T.\text{value} \times F.\text{value}$ }
- { $T.\text{value} = F.\text{value}$ }
- { $F.\text{value} = \text{id}.\text{value}$ }

▶ Productions

1. Construct the parse tree and associate attribute to every Non-terminal.
2. During the (top-bottom & left right) traversal, whenever we come across a reduction, we are to go to the production rule & carry out the associated semantic action.

Infix to Postfix(Top Down Parsing)

Expression: $2 + 3 \times 4 = 14$



O/P: 2 3 4 x +

▶ Grammar

- i. $E \rightarrow E + T$
- ii. $E \rightarrow T$
- iii. $T \rightarrow T * F$
- iv. $T \rightarrow F$
- v. $F \rightarrow id$

▶ Semantic Rules

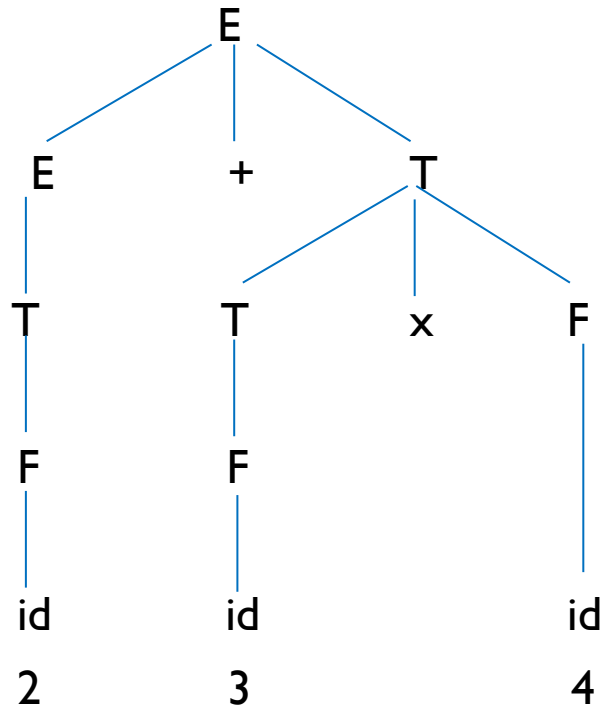
- 1. { printf("+"); }
- 2. { }
- 3. { printf("x"); }
- 4. { }
- 5. { printf(id.value); }

Procedure:

- 1. During Top-down parsing, the semantic action is first given a value & it's considered as the rightmost element in the R.H.S. of the production.
- 2. During traversal, action its taken whenever any semantic action number is encountered.

Infix to Postfix(Bottom Up Parsing)

Expression: $2 + 3 \times 4 = 14$



O/P: 2 3 4 x +

▶ Grammar

- i. $E \rightarrow E + T$
- ii. $E \rightarrow T$
- iii. $T \rightarrow T * F$
- iv. $T \rightarrow F$
- v. $F \rightarrow id$

▶ Semantic Rules

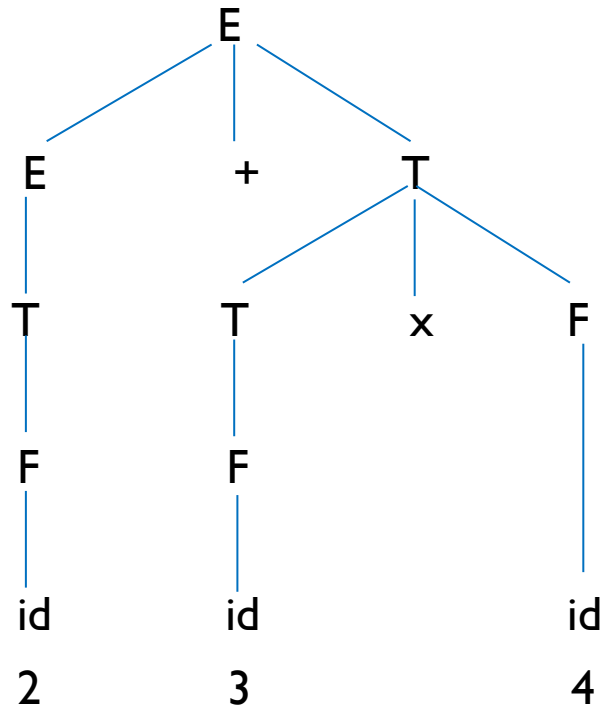
- 1. { printf("+"); }
- 2. { }
- 3. { printf("x"); }
- 4. { }
- 5. { printf(id.value); }

Procedure:

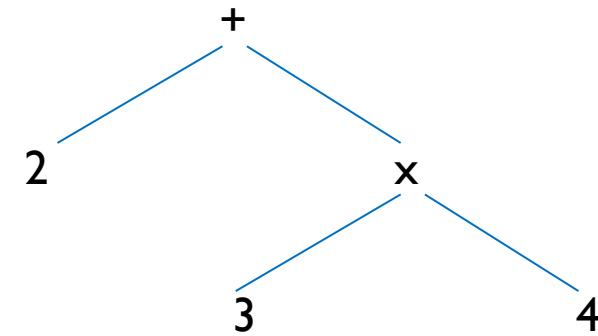
- 1. Bottom up parsers mainly focus on reduction.
- 2. During reduction, the appropriate production is chosen and the associated action is performed.

Parse Tree and Abstract Syntax Tree

► 2+3x4



Parse Tree



Abstract Syntax Tree

Construction of AST

► Grammar

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow \text{id}$

► Semantic Rules

```
{ E.nptr = mknode(E.nptr, '+', T.nptr); }  
{ E.nptr = T.nptr; }  
{ T.nptr = mknode(T.nptr, 'x', F.nptr); }  
{ T.nptr = F.nptr; }  
{ F.nptr = mknode(null, id.value, null); }
```

Construction of AST

Grammar

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow \text{id}$

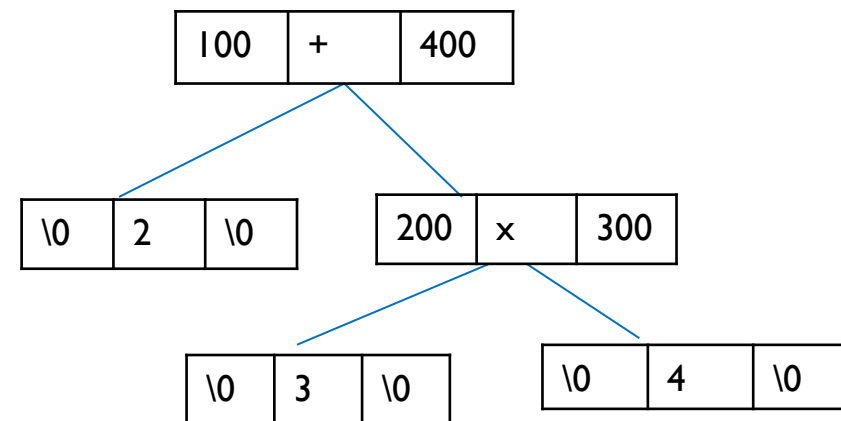
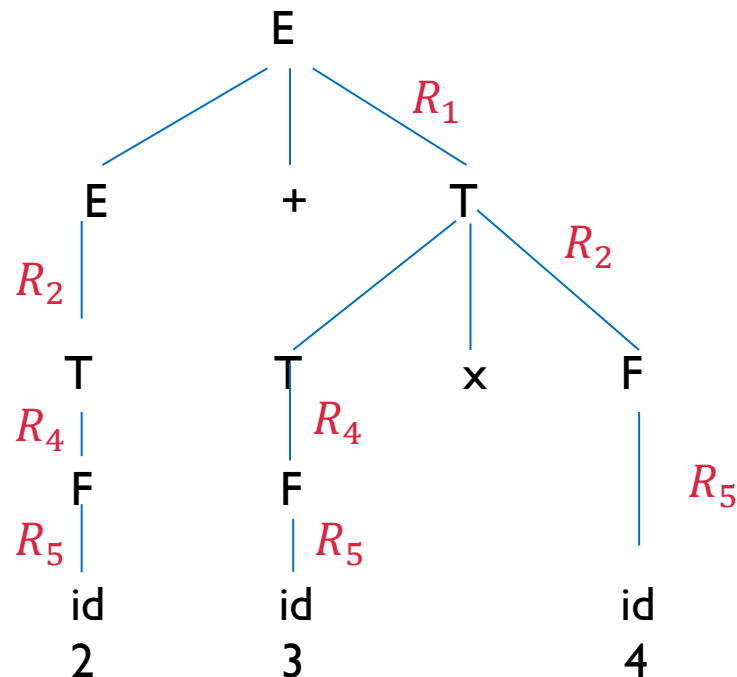
Semantic Rules

1. $\{ E.\text{nptr} = \text{mknode}(E.\text{nptr}, '+', T.\text{nptr}); \}$
2. $\{ E.\text{nptr} = T.\text{nptr}; \}$
3. $\{ T.\text{nptr} = \text{mknode}(T.\text{nptr}, 'x', F.\text{nptr}); \}$
4. $\{ T.\text{nptr} = F.\text{nptr}; \}$
5. $\{ F.\text{nptr} = \text{mknode}(\text{null}, \text{id.value}, \text{null}); \}$

Actions

Expression: 2+3x4

1. $R_5: F.\text{nptr} = 100$
2. $R_4: T.\text{nptr} = F.\text{nptr} \quad T.\text{nptr} = 100$
3. $R_2: E.\text{nptr} = T.\text{nptr} \quad E.\text{nptr} = 100$
4. $R_5: F.\text{nptr} = 200$
5. $R_4: T.\text{nptr} = F.\text{nptr} \quad T.\text{nptr} = 200$
6. $R_5: F.\text{nptr} = 300$
7. $R_3: T.\text{nptr} = 400$
8. $R_1: E.\text{nptr} = 500$



Type Checking

Expression: $(2+3)==8$

▶ Grammar

1. $E \rightarrow E_1 + E_2$
2. $E \rightarrow E_1 == E_2$
3. $E \rightarrow (E_1)$
4. $E \rightarrow \text{num}$
5. $E \rightarrow \text{True}$
6. $E \rightarrow \text{False}$

▶ Semantic Rules

- { if($(E_1.\text{type} == E_2.\text{type}) \&\& (E_1.\text{type} == \text{int})$) then $E.\text{type} = \text{int}$ else ERROR; }
- { if($(E_1.\text{type} == E_2.\text{type}) \&\& (E_1.\text{type} == \text{int} | \text{bool})$) then $E.\text{type} = \text{bool}$ else ERROR; }
- { $E.\text{type} = E_1.\text{type}$; }
- { $E.\text{type} = \text{int}$; }
- { $E.\text{type} = \text{bool}$; }
- { $E.\text{type} = \text{bool}$; }

Type Checking

▶ Grammar

1. $E \rightarrow E_1 + E_2$
2. $E \rightarrow E_1 == E_2$
3. $E \rightarrow (E_1)$
4. $E \rightarrow \text{num}$
5. $E \rightarrow \text{True}$
6. $E \rightarrow \text{False}$

▶ Actions

- | | |
|------------------------------------|--------------------|
| 1. $R_4: E \rightarrow \text{num}$ | E_1 's type: int |
| 2. $R_4: E \rightarrow \text{num}$ | E_2 's type: int |
| 3. $R_1: E \rightarrow E + E$ | E's type: int |
| 4. $R_3: E \rightarrow (E)$ | E_1 's type: int |
| 5. $R_4: E \rightarrow \text{num}$ | E_2 's type: int |
| 6. $R_2: E \rightarrow E == E$ | E's type: bool |

▶ Semantic Rules

{ if(($E_1.\text{type} == E_2.\text{type}$)&&(($E_1.\text{type} == \text{int}$)))then E.type=int else ERROR; }

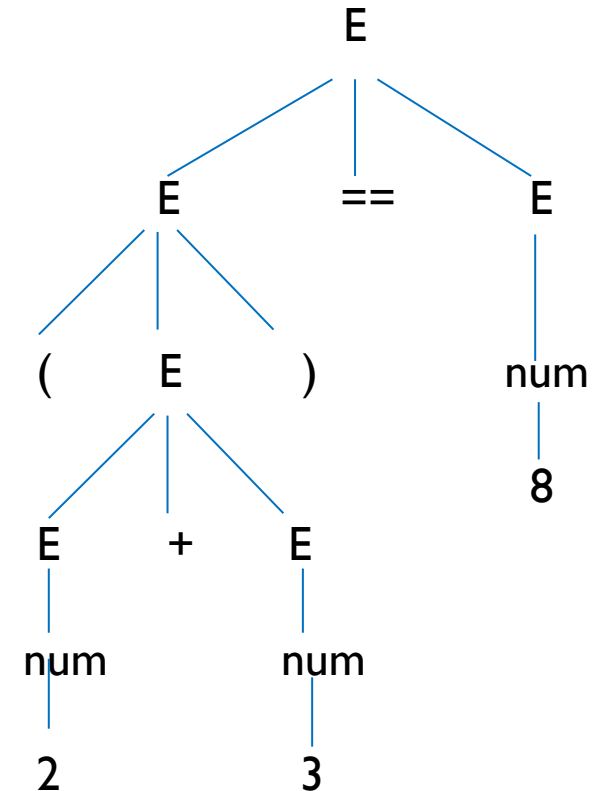
{ if(($E_1.\text{type} == E_2.\text{type}$)&&(($E_1.\text{type} == \text{int}|\text{bool}$)))then E.type=bool else ERROR; }

{ E.type = $E_1.\text{type}$; }

{ E.type = int; }

{ E.type = bool; }

{ E.type = bool; }



Types of Attributes

- ▶ In SDTs, every variable (non terminal) in the production rules are associated with attributes.
 1. Synthesized Attributes: Attributes deriving values from the non terminal children(mentioned by the production rules). Eg. $A \rightarrow BCD$ then $A.attr = f(B.attr, C.attr, D.attr)$. The $A.attr$ is synthesized attribute.
 2. Inherited Attributes: Attributes deriving values from the parent and siblings of the non terminal (mentioned by the production rules). Eg. $A \rightarrow BCD$ then $C.attr = f(A.attr) | f(B.attr) | f(D.attr)$. The $C.attr$ is inherited attribute.

Example

Let the grammar be

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$E' \rightarrow \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow \varepsilon$$

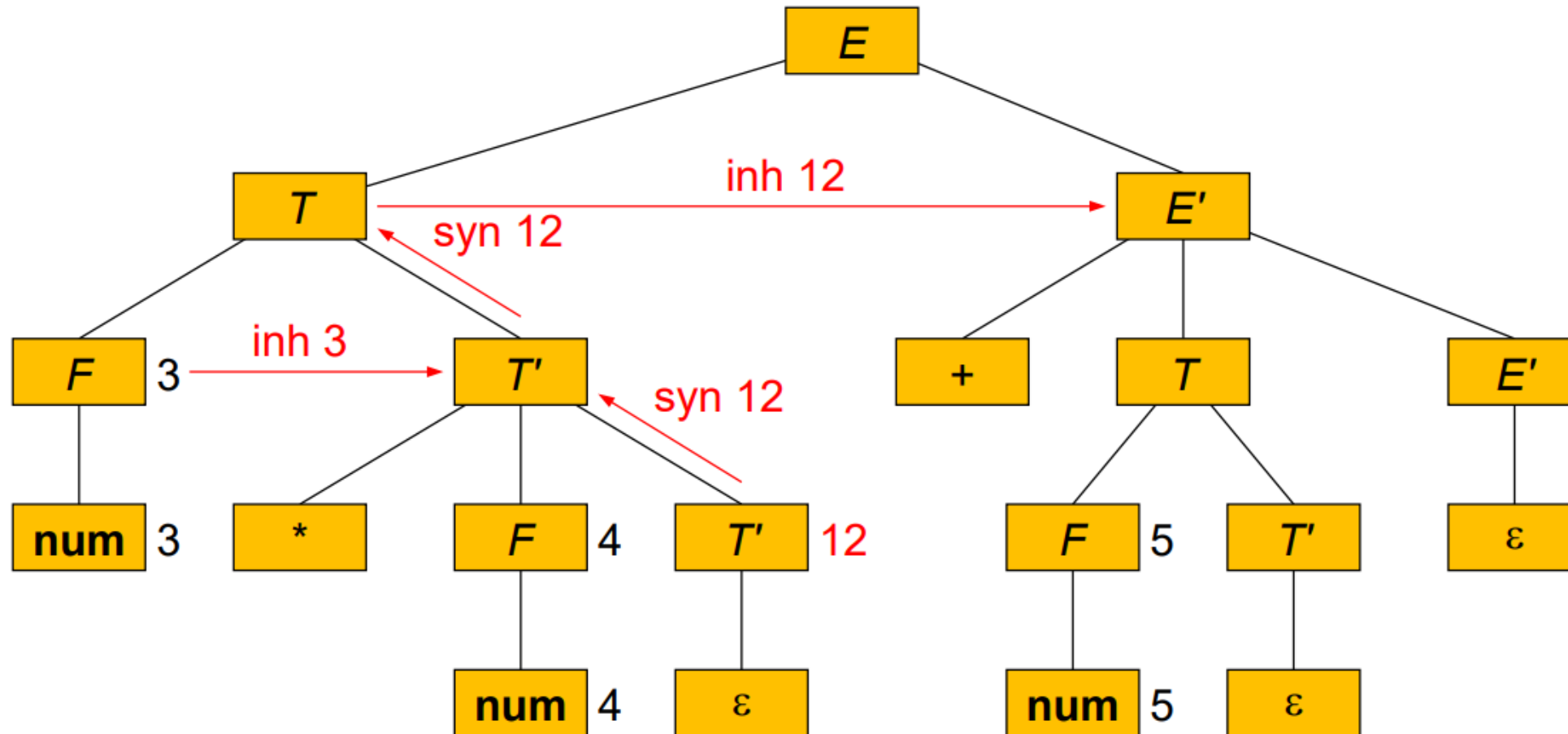
$$F \rightarrow (E)$$

$$F \rightarrow \text{num}$$

Example

- ▶ How does T (in the production $T \rightarrow F T'$) get its value?
- ▶ It must multiply 3 and 4 to get 12.
- ▶ So, first T' inherits 3 from F .
- ▶ Then, in the production $T' \rightarrow * F T_1'$, T_1' inherits 12 from T' and F .
- ▶ Then, T' turns around and synthesizes 12 from T_1' .
- ▶ Then, back in the production $T \rightarrow F T'$, T synthesizes 12 from T'

Example



Types of SDTs

S-attributed SDT	L-attributed SDT
Uses only synthesized attribute	Uses both synthesized and inherited attributes. Each inherited attribute is restricted to inherit either from parent or left sibling(s) only. E.g. $A \rightarrow XYZ$ $\{Y.attr=A.attr, Y.attr=X.attr, Y.attr \neq Z.attr\}$
Semantic actions are placed at the right most end of the productions. Eg. $A \rightarrow BC\{\}$	Semantic actions can be placed anywhere in the RHS. Eg. $A \rightarrow \{ \} BC D \{ \} E FG \{ \}$
Attributes are evaluated during bottom-up parsing.	Attributes are evaluated by traversing the parse tree depth first, left to right.

Dependency Graph

- ▶ Used to represent flow of information among attributes in a parse tree
- ▶ Helps to determine the evaluation order for the attributes in a parse tree
- ▶ Main aim is to help compiler check for various types of dependencies between statements in order to prevent them from being executed in the incorrect sequence
- ▶ While annotated parse tree show the values of the attributes, dependency graph determines how these values can be computed.

Dependency Graph (exercise)

Draw dependency graph and write the topological sort for the input **3*4** for following syntax-directed definition of a simple desk calculator

Input: **3*4**

Production

$L \rightarrow E \mathbf{n}$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \mathbf{digit}$

Semantic Rules

$\text{print}(E.\text{val})$

$E.\text{val} = E_1.\text{val} + T.\text{val}$

$E.\text{val} = T.\text{val}$

$T.\text{val} = T_1.\text{val} * F.\text{val}$

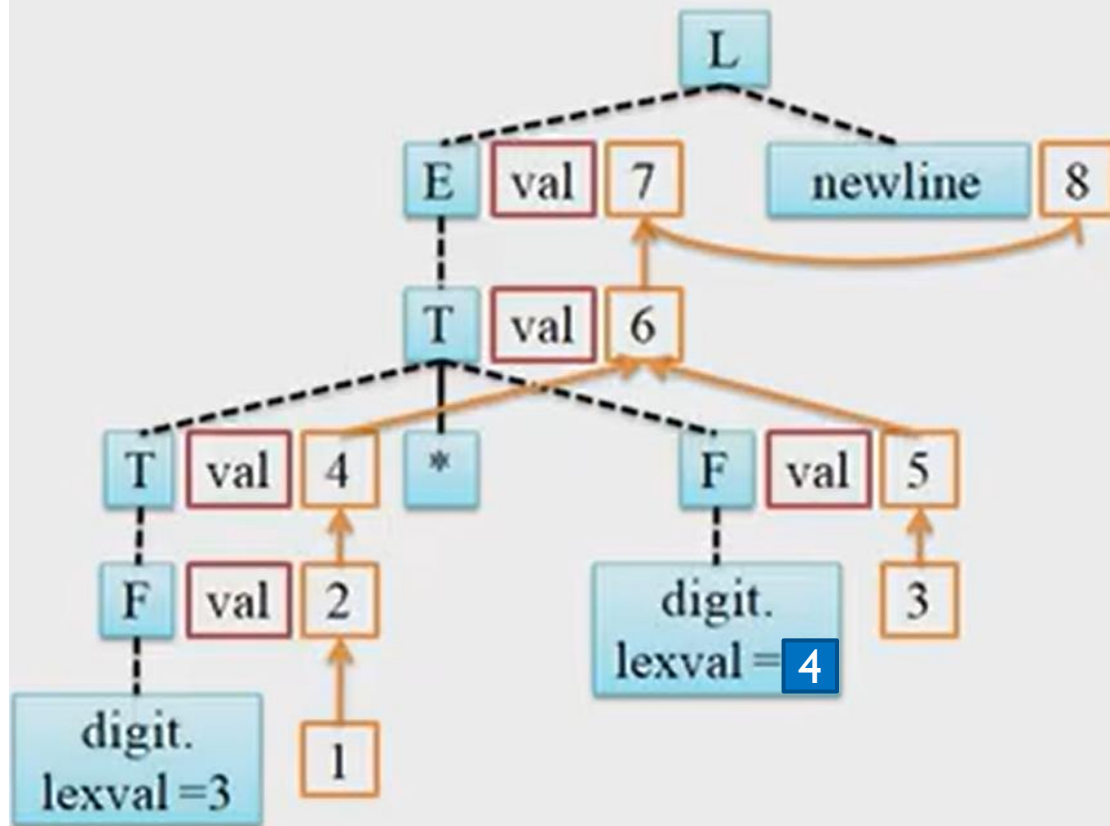
$T.\text{val} = F.\text{val}$

$F.\text{val} = E.\text{val}$

$F.\text{val} = \mathbf{digit}.\text{lexval}$

Dependency Graph

Draw dependency graph and write the topological sort for the input $3*4$ for following syntax-directed definition of a simple desk calculator



Input: $3*4$

Production

$L \rightarrow E \mathbf{n}$
 $E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow \mathbf{digit}$

Semantic Rules

$\text{print}(E.\text{val})$
 $E.\text{val} = E_1.\text{val} + T.\text{val}$
 $E.\text{val} = T.\text{val}$
 $T.\text{val} = T_1.\text{val} * F.\text{val}$
 $T.\text{val} = F.\text{val}$
 $F.\text{val} = E.\text{val}$
 $F.\text{val} = \mathbf{digit}.\text{lexval}$

Assignment

- ▶ Draw the dependency graph for $a - 4 + c$ in top-down parsing